MERGE SORT

ANALYSIS

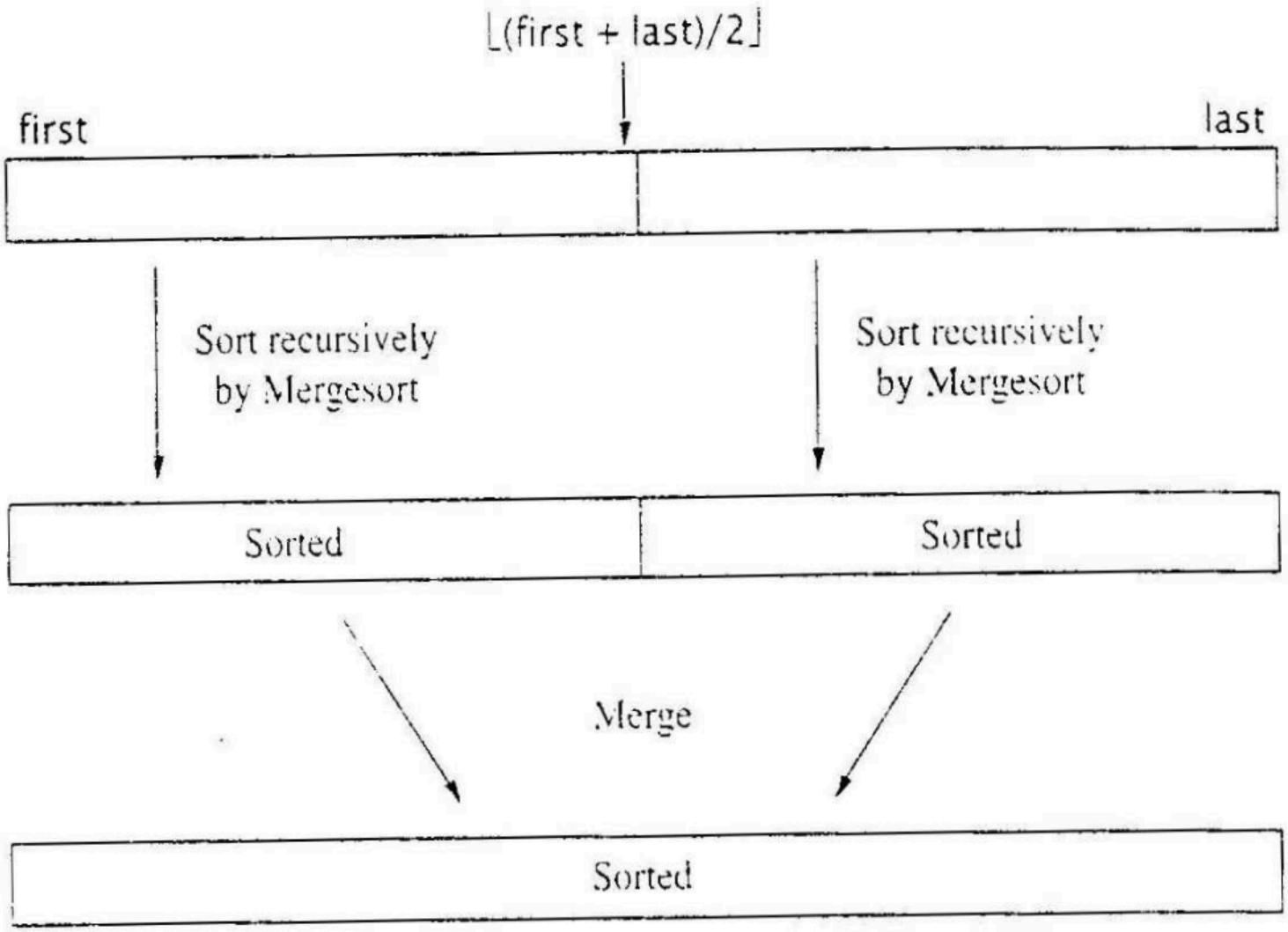
Divide and Conquer

- **It is often easier to *solve several small instances* of a problem than one large one.**
  - *divide* the problem into smaller instances of the same problem
  - solve (*conquer*) the smaller instances recursively
  - *combine* the solutions to obtain the solution for original input

- Solve(I)
  - **n = size(I)**
  - **if (n <= smallsize)**
    - solution = directlySolve(I);
  - **else**
    - divide I into I1, , Ik.
    - for each i in {1, , k}
      - Si = solve(Ii);
    - solution = combine(S1, , Sk);
  - **return solution;**

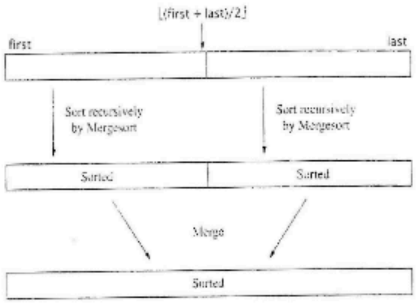
Mergesort : Using Divide and Conquer

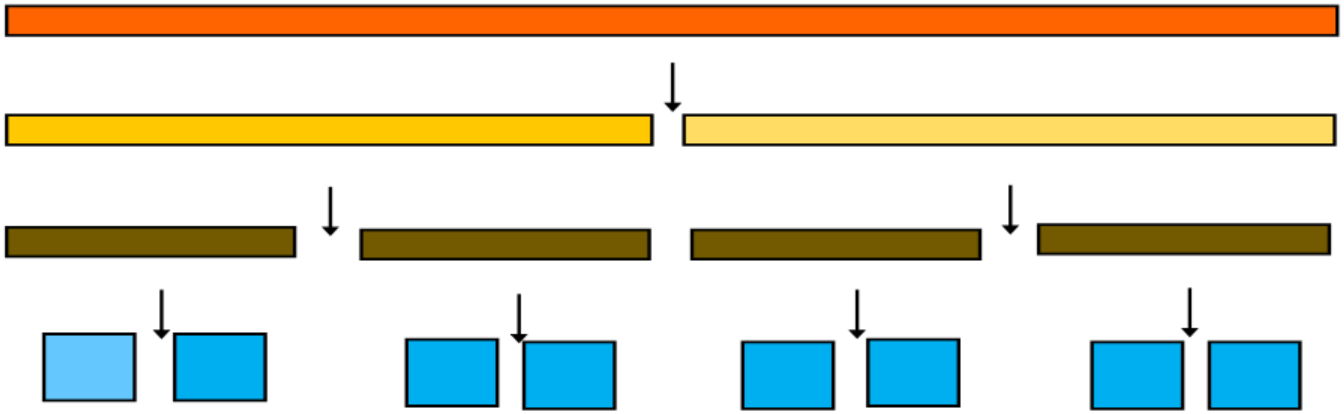
Mergesort Strategy

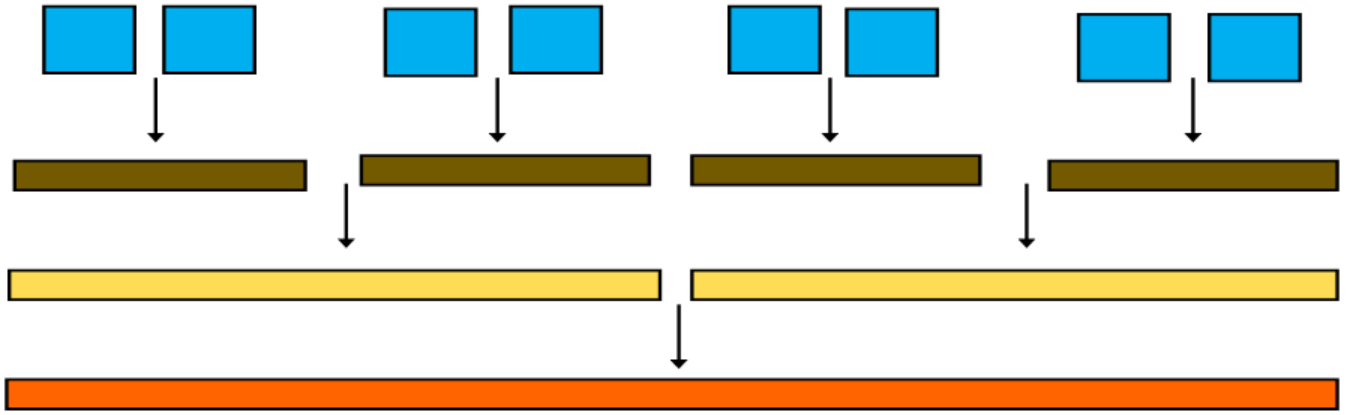


Using Divide and Conquer: Mergesort

Mergesort Strategy







Algorithm: Mergesort

Input: Array A and indexes first (p) and last (r).

Output: A[first], …, A[last] is sorted rearrangement of the same elements

Pseudocode for MergeSort

```
MERGESORT(A, p, r)
if  p< r
    then q← floor[(p+r)/2]
         MERGESORT(A, p, q)
    MERGESORT(A, q+1, r)
      MERGE(A,p,q,r)
```

MERGE(A,p,q,r) - Merging Sorted Sequences

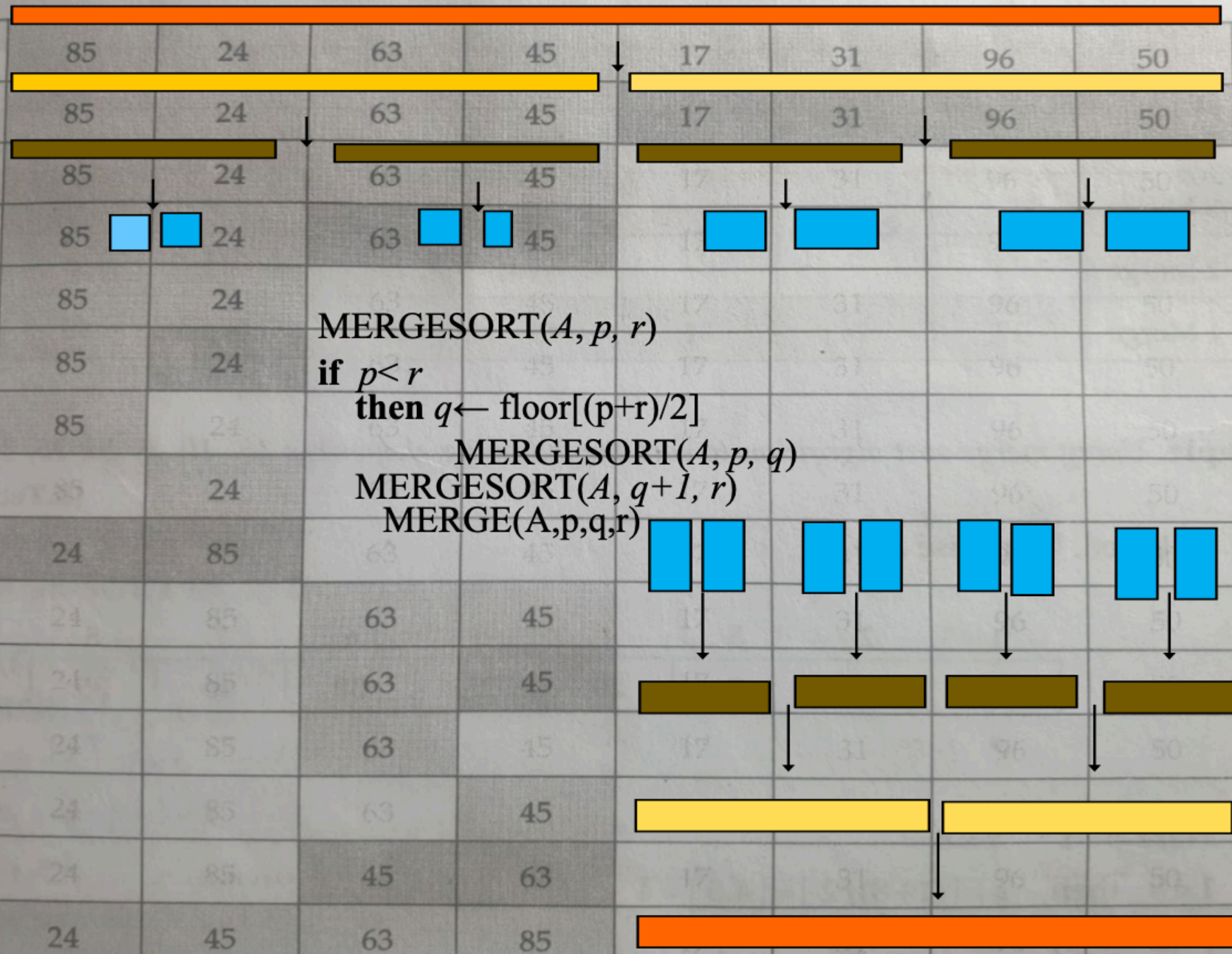
- **Problem:**
  - ➢**Given two sequences A and B sorted in nondecreasing order,**
  - ➢**merge them to create one sorted sequence C**

- Strategy:
  - ✈**determine the first item in C:**
  - ✓It is the minimum between the first items of A and B.
  - ✓Suppose it is the first items of A. Then, rest of C consisting of merging rest of A with B.

Example: A (12, 34, 38, 78) B(2, 56, 87, 90, 98)

C(2, 12, 34, 38, 56, 78, 87, 90, 98)

Example of Merge Sort



MERGESORT(A, p, r)

if $p < r$

then $q \leftarrow$ floor[(p+r)/2]

MERGESORT(A, p, q)

MERGESORT(A, $q+1$, r)

MERGE(A,p,q,r)

Merge(A,p,q,r)

{

- n1=q – p + 1
- n2=r - q
- Let L[1..n1] & R[1..n2] be new array
- for (i= 0 to n1-1)
  - L[i]= A[p+i]
- for (j= 0 to n2-1)
  - R[j]= A[(q+1)+j]

- i=0, j=0
- for (k= p to r)
- if (L[i] <= R[j])
  - A[k] = L[i]
  - i =i +1
- else
  - A[k] = R[j]
  - j=j+1

}

MERGE SORT ANALYSIS

```
Merge(A,p,q,r)
•{
•n1=q – p + 1
•n2=r - q
•Let L[1..n1] & R[1..n2] be new array
•for ( i= 0 to n1-1 )
•       L[i]= A[p+i]
•for ( j= 0 to n2-1 )
•       R[j]= A[q+1+j]

•i=0, j=0
•for ( k= p to r )
•if (L[i]  <= R[j])
•       A[k] = L[i]
•       i =i +1
•else
•       A[k] = R[j]
•       j=j+1
•}
```

→ **O(n)**

MERGESORT(*A*, *p*, *r*)
if *p*< *r*
then *q*← floor[(p+r)/2]
MERGESORT(*A*, *p*, *q*)
MERGESORT(*A*, *q+1*, *r*)
MERGE(A,p,q,r)

Total Cost = Divide + conquer + merge
T(n) = C1 + 2T(n/2) + Cn
 O(n log n)

Space Complexity

Calculation of stack size :

In general, **stack size will be levels in a tree**, which is (log n),

Memory Requirement

- **Each merge-sort consumes = O (log n)**
- **Merge Consumes = O(n)**

Total space rqmnts = O (log n) + O(n) = O(n)

```
MERGESORT(A, p, r)
if p< r
   then q← floor[(p+r)/2]
                MERGESORT(A, p, q)
   MERGESORT(A, q+1, r)
     MERGE(A,p,q,r)
```

This is also called out-of-space / out-place algorithms as its needs extra array to do partial processing.
Extra space used by an algorithm depends on the input size.

It requires 'n' extra cells to copy elements from two sorted arrays into one, therefore each merge procedure requires O(n) memory. Since this memory is **reused** 'log n' times.

The effective memory requirement is **O(n)**

Time Complexity

```
MERGESORT(A, p, r)
if  p< r
    then q← floor[(p+r)/2]
         MERGESORT(A, p, q)
     MERGESORT(A, q+1, r)
      MERGE(A,p,q,r)
```

Divide n Conquer Notation

[T(n)= Divide + Recursion+ Merge]

$$T(n) = O(1) + T(n/2) + T(n/2) + O(n)$$
$$= (n) + 2\,T(n/2) \quad \text{[By masters theorem]}$$
$$= \Theta(n \log n)$$

There is no worst scenario as it works same for ascending or descending order

Calculate the cost :

Given n elements, merge them using merge procedure to get one sorted list of n elements.

Calculate the cost :

Given n elements, merge them using merge procedure to get one sorted list of n elements.

- Each merge procedure time to merge two sorted list into one and moving to next higher level = O(n)

- Total levels to reach to the one sorted list = log n

➔ **Total time = O(n) * log n = O (n log n)**

Given “log n” sorted list each of size “n/log n”

what is the total time required to merge them into single sorted list

Given “log n” sorted list each of size “n/log n”

what is the total time required to merge them into single sorted list

N elements each of size 1 will take **n*1 = O(n)**	(log n) elements each of size n/logn will take **(log n)* n/(log n) = O(n)**
For n elements, number of levels will be **log n**	For (log n) elements, number of levels will be **log (log n)**
Total Cost = cost of merge at each level * total levels = **O(n) * log n = O (n log n)**	Total Cost = cost of merge at each level * total levels = **O(n) * log (log n) = O (n log (log n))**

“n” strings each of length “n” are given.

-What will be the time taken to sort them ?

“n” strings each of length “n” are given.

-What will be the time taken to sort them ?

Cost of merging n elements(size 1) = O(n)

Hence, cost of merging n strings, each of size n

$$= O(n) * O(n) = O(n^2)$$

Total levels = (log n)

therefore,

Total Cost T(n) = $O(n^2) * \log n = \mathbf{O(n^2 \log n)}$

Assume that size k problem could be independently solved (instead of 1)

n numbers (k<n) to be sorted

Compute the running time of Merge sort ?

Assume that size k problem could be independently solved (instead of 1)

Compute the running time of Merge sort ?

- Number of elements of size k to be merged (n/k)
- Cost of merging k elements of size (n/k) = O (n)
- Total numbers of levels = log (n/k)

➜ Total cost = O (n) * log (n/k)

= **O(n log (n/k))**

Video Lecture of the topic can be found at

Quick Sort Analysis for various partitioning scenarios
https://youtu.be/gCP06CTpEaE

Merge Sort Algorithm

https://www.youtube.com/watch?v=RH0p7DYyMn4

Merge Procedure illustrative Example
https://www.youtube.com/watch?v=WzDq-WQALy0

Understanding Space Complexity
https://youtu.be/HEZdrU8ZPIk

Merge Sort Complexity Analysis
https://www.youtube.com/watch?v=x1dJx89Elf0

https://cse.iitkgp.ac.in/~abhij/course/theory/Fall20/slides/comparison-based-sorting-ca/